

PROJECT REPORT

ON

EV CHARGING SLOT SCHEDULER

An Algorithm-Based Web Prototype for Efficient EV Charging Slot Allocation

Submitted in partial fulfillment of the requirements
for the B.Tech Degree

Submitted by

M.LAVANYA [25MVCSMR0660]

P.AKSHAYA[25MVCSMRO669]

P.VANDHANA[25MVCMR0666]

P.SRINITHYA[25MVCSMR0670]

Department : CSE(AIML)

University Name: MALLAREDDY TECHNICAL CAMPUS

[Academic Year 2026–2027]

CERTIFICATE

This is to certify that the project entitled “EV Charging Slot Scheduler” is a bonafide project work carried out by [Student Name], [Roll Number], under the guidance of [Guide Name], in partial fulfillment of the requirements for the Bachelor of Technology degree.

The project demonstrates the use of algorithmic techniques such as Priority Queue, Greedy Scheduling, and Job Sequencing to allocate charging slots to electric vehicles based on urgency, departure time, and charging duration.

DECLARATION

I hereby declare that the project report entitled “EV Charging Slot Scheduler” is my original work carried out as part of my B.Tech academic project. The work presented in this report has been developed for educational and prototype purposes. All major technical concepts and external resources used in the project have been appropriately acknowledged.

M.LAVANYA [25MVCSMR0660]

P.AKSHAYA[25MVCSMRO669]

P.VANDHANA[25MVCMR0666]

P.SRINITHYA[25MVCSMR0670]

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, faculty members, and department for their support and guidance during the development of this project.

I also thank my classmates, friends, and family for their encouragement and support. Finally, I am grateful for the software tools and learning resources that helped me understand Flask, Python, PostgreSQL, and scheduling algorithms.

ABSTRACT

The EV Charging Slot Scheduler is a web-based prototype designed to assign charging slots to electric vehicles in an organized and algorithm-driven manner. As the number of electric vehicles increases, charging stations may face situations where several vehicles request charging at the same time. A simple first-come-first-served approach may not always handle urgent vehicles efficiently.

The proposed system considers three important factors: vehicle urgency, expected departure time, and charging duration. A Priority Queue is used to organize vehicles according to urgency and departure time. Greedy scheduling is used to select feasible charging opportunities, while a Job Sequencing approach demonstrates another way of assigning vehicles before their deadlines. Python and Flask are used for the application logic and web interface, while PostgreSQL is used to store vehicle and scheduling information.

The prototype displays scheduling results, vehicle status, charging duration, and a charging timeline. It is intended as an educational and algorithm-focused demonstration and can be extended to support multiple charging stations, real-time vehicle data, notifications, and advanced optimization methods.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Existing System
5. Proposed System
6. System Requirements
7. Technologies Used
8. System Architecture
9. Methodology
10. Algorithms Used
11. Database Design
12. Module Description
13. Implementation
14. Sample Prototype Output
15. Advantages
16. Limitations
17. Future Scope
18. Conclusion
19. References

1. INTRODUCTION

Electric vehicles are becoming an important part of modern transportation. One of the practical challenges in EV infrastructure is managing charging requests when multiple vehicles need charging within a limited period.

The EV Charging Slot Scheduler provides a simple solution to this problem by using scheduling algorithms. Instead of treating every vehicle in the same way, the system considers urgency, departure time, and charging duration before assigning a slot.

The project is implemented as a web prototype using Python and Flask. PostgreSQL stores the submitted vehicle information and scheduling status. The prototype helps demonstrate how data structures and algorithms can be applied to a real-world EV charging problem.

2. PROBLEM STATEMENT

Multiple electric vehicles may arrive at a charging station with different urgency levels, departure times, and charging requirements. If charging slots are assigned without considering these factors, an important vehicle may wait too long or may not finish charging before its departure.

The problem is to develop a system that accepts vehicle information and assigns charging slots using algorithmic scheduling techniques. The system should identify feasible schedules, show which vehicles are scheduled or not scheduled, and store the results for later viewing.

3. OBJECTIVES

- To develop a simple web-based EV charging slot scheduler.
- To prioritize vehicles according to urgency and departure time.
- To apply Priority Queue and Greedy scheduling concepts.
- To demonstrate Job Sequencing for deadline-based allocation.
- To check whether a vehicle can complete charging before its departure time.
- To display scheduling results and a charging timeline.
- To store vehicle and scheduling information in PostgreSQL.
- To provide a prototype that can be extended for real charging stations.

4. EXISTING SYSTEM

In a basic charging station, charging requests may be handled manually or using simple first-come-first-served rules. Such approaches are easy to understand but may not consider different urgency levels or deadlines.

The major challenges of a simple approach include:

- Urgent vehicles may have to wait behind less urgent vehicles.
- Departure time may not be considered.
- Charging duration can affect the feasibility of later requests.
- Manual scheduling becomes difficult as the number of vehicles increases.

5. PROPOSED SYSTEM

The proposed system provides an algorithm-based scheduling process. The user enters the vehicle ID, urgency, departure time, and charging duration. The system processes these values and generates a schedule.

The prototype uses a Priority Queue to process vehicles based on urgency and departure time. A Greedy strategy selects a feasible charging opportunity. A Job Sequencing module is also provided to demonstrate deadline-based scheduling.

The system saves vehicle information and the generated status in PostgreSQL. A separate page displays previously saved vehicles.

6. SYSTEM REQUIREMENTS

6.1 Hardware Requirements

Component	Requirement
Processor	Intel/AMD processor or equivalent
RAM	Minimum 4 GB
Storage	At least 500 MB free space
Display	Standard monitor/laptop display
Network	Not required for local prototype; useful for deployment

6.2 Software Requirements

Software	Purpose
Python	Programming language
Flask	Web application framework
PostgreSQL	Database
psycopg2-binary	Python PostgreSQL connector
HTML/CSS	User interface
Web Browser	Run and test the prototype
VS Code / IDE	Development environment

7. TECHNOLOGIES USED

Technology	Role
Python	Application and scheduling logic
Flask	Web server and routing
PostgreSQL	Persistent storage of vehicle data
HTML	Page structure and forms
CSS	Dashboard and timeline styling
Priority Queue	Urgency-based vehicle processing
Greedy Algorithm	Feasible slot selection
Job Sequencing	Deadline-based scheduling demonstration

8. SYSTEM ARCHITECTURE

The system follows a simple three-layer flow:

1. User Interface: The user enters vehicle details through the Flask web page.

2. Application Layer: Flask receives the data and sends it to the scheduling algorithms.
3. Database Layer: PostgreSQL stores vehicle details and their scheduling status.

Overall flow:

User → Flask Web Interface → Scheduling Algorithms → Result Dashboard →
PostgreSQL

9. METHODOLOGY

The scheduler follows these general steps:

4. Collect vehicle ID, urgency, departure time, and charging duration.
5. Validate and convert the input values.
6. Insert vehicles into the Priority Queue using urgency and departure time.
7. Process vehicles in priority order.
8. Check whether the vehicle can finish charging before its departure time.
9. Assign a charging interval when the request is feasible.
10. Mark the vehicle as scheduled or not scheduled.
11. Generate a charging timeline for scheduled vehicles.
12. Run the Job Sequencing approach for comparison.
13. Store vehicle information and status in PostgreSQL.
14. Display the results on the dashboard.

10. ALGORITHMS USED

10.1 Priority Queue

A Priority Queue is a data structure in which elements are processed according to priority rather than simply according to arrival order. In this project, urgency is the main priority. Departure time is also used when ordering vehicles with similar priority.

Example: If EV01 has urgency 9 and EV02 has urgency 5, EV01 is considered before EV02.

10.2 Greedy Algorithm

The Greedy approach makes a suitable decision at each step without trying every possible complete schedule. In this prototype, the next vehicle is considered according to priority and the system checks whether enough time is available before its departure.

A vehicle is marked Scheduled when its charging can finish within the available time. Otherwise, it is marked Not Scheduled.

10.3 Job Sequencing

Job Sequencing is a deadline-based scheduling concept. In this project, vehicles are processed by urgency and the algorithm attempts to place each vehicle in the latest feasible interval before its departure deadline. This helps demonstrate how deadline-based scheduling can be applied to charging requests.

10.4 Algorithm Flow

15. Start

16. Read vehicle details
17. Create priority queue
18. Select the next high-priority vehicle
19. Check available charging time
20. Schedule if feasible
21. Otherwise mark as not scheduled
22. Update charging timeline
23. Store status in database
24. Display results
25. Stop

11. DATABASE DESIGN

The project uses PostgreSQL to store vehicle and scheduling information.

Field	Data Type	Description
id	SERIAL PRIMARY KEY	Unique database record ID
vehicle id	VARCHAR(50)	Vehicle identifier
urgency	INTEGER	Urgency value, typically 1–10
departure	INTEGER	Departure deadline/slot
duration	INTEGER	Required charging duration
status	VARCHAR(30)	Scheduled or Not Scheduled

Table name: vehicles

12. MODULE DESCRIPTION

Module	Description
app.py	Flask routes, form processing, dashboard, and database interaction
scheduler.py	Priority Queue, Greedy scheduling, and Job Sequencing logic
database.py	PostgreSQL connection and database operations
index.html	Main scheduler interface and results dashboard
vehicles.html	Displays saved vehicles from PostgreSQL
style.css	Styles the dashboard and charging timeline

13. IMPLEMENTATION

13.1 Input Module

The user enters details for each EV. The main fields are Vehicle ID, Urgency, Departure Time, and Charging Duration.

13.2 Scheduling Module

The scheduling module receives the vehicle list and generates the Priority Queue/Greedy result. Each vehicle receives a status and, when scheduled, a start and end slot.

13.3 Job Sequencing Module

The Job Sequencing function sorts vehicles by urgency and attempts to allocate the latest available interval before the vehicle deadline.

13.4 Database Module

The database module connects the Flask application to PostgreSQL. Vehicle records are inserted after scheduling, and the saved records can be viewed through the Saved Vehicles page.

13.5 Dashboard Module

The dashboard displays total vehicles, scheduled vehicles, not scheduled vehicles, total charging duration, scheduling results, and the charging timeline.

14. SAMPLE PROTOTYPE OUTPUT

The following sample input can be used to demonstrate the prototype:

Vehicle	Urgency	Departure	Duration
EV01	9	8	2
EV02	6	6	2
EV03	8	10	3

The dashboard then displays the generated schedule. Scheduled vehicles are shown with their assigned start and end slots. Vehicles that cannot be accommodated before their deadlines are shown as Not Scheduled.

The prototype also provides a Charging Station Timeline, where each scheduled vehicle is represented by its charging interval.

14.1 Screenshot – Home / Input Page

[Insert screenshot of the EV Charging Slot Scheduler input page here.]

14.2 Screenshot – Scheduling Results

[Insert screenshot of the scheduling results/dashboard here.]

14.3 Screenshot – Charging Timeline

[Insert screenshot of the charging station timeline here.]

14.4 Screenshot – Saved Vehicles

[Insert screenshot of the PostgreSQL saved vehicles page here.]

15. ADVANTAGES

- Simple and easy-to-use web interface.
- Considers urgency, departure time, and charging duration.
- Demonstrates multiple scheduling concepts in one project.
- Automatically identifies infeasible requests.
- Stores vehicle records using PostgreSQL.
- Provides a visual charging timeline.
- Can be extended to larger EV charging systems.

16. LIMITATIONS

- The current prototype models a simplified charging station.
- Time values are represented as integer slots rather than real clock times.
- The prototype does not yet use live charging-station data.
- Multiple physical charging stations are not fully modeled.
- Job Sequencing is adapted for variable charging durations and is intended as an educational scheduling demonstration.
- The prototype does not include payment, user accounts, or mobile notifications.

17. FUTURE SCOPE

- Support multiple charging stations and connectors.
- Use actual clock times and estimated charging completion times.
- Add real-time availability of charging points.
- Allow users to book and cancel charging slots.
- Add notifications for charging start and completion.
- Add authentication for drivers and station administrators.
- Use live EV battery percentage and estimated range.
- Add advanced optimization methods for large numbers of vehicles.
- Deploy the system on a cloud server for remote access.
- Provide analytics such as station utilization and waiting time.

18. CONCLUSION

The EV Charging Slot Scheduler demonstrates how data structures and scheduling algorithms can be applied to a practical electric-vehicle charging problem. The system considers urgency, departure time, and charging duration to generate a feasible schedule.

The project combines Python, Flask, PostgreSQL, Priority Queue, Greedy scheduling, and Job Sequencing in one working prototype. The dashboard and charging timeline make the algorithm results easier to understand. Although the current implementation is a simplified prototype, it provides a foundation for developing a more advanced real-world EV charging management system.

19. REFERENCES

- Python Documentation – Python programming language and standard library.
- Flask Documentation – Web application development with Flask.
- PostgreSQL Documentation – Relational database management system.
- Cormen, Leiserson, Rivest and Stein, Introduction to Algorithms – Scheduling and data structure concepts.
- General academic resources on Priority Queues, Greedy Algorithms, and Job Sequencing.

THANK YOU

EV Charging Slot Scheduler